Vulnerability Assessment - Study Guide

Concepts, Scoring, Tools, Reporting & Lab Walkthroughs

Ethical Hacking / Network Security Study Notes
November 2026

1. Vulnerability Assessment Concepts
2. Vulnerability Assessment Solutions
3. Vulnerability Scoring Systems
4. Vulnerability Assessment Tools
5. Vulnerability Assessment Reports
6. Lab Walkthrough: Perform Vulnerability Research in CWE
7. Lab Walkthrough: Perform Vulnerability Research in CVE
8. Lab Walkthrough: Perform Vulnerability Analysis Using OpenVAS
9. Lab Walkthrough: Perform Vulnerability Scanning Using Nessus
10. Lab Walkthrough: Perform Web Server/Application Vulnerability Scanning Using Nikto
Quick Revision Summary

> ***Scope & Ethics Notice*** *This guide is for educational / certification-study purposes (e.g., CEH-style exam prep). Vulnerability scanning tools described here (OpenVAS, Nessus, Nikto) must only be run against systems you own or have explicit written authorization to test — scanning systems without permission is illegal in most jurisdictions, even when using "defensive" tools.*

1. Vulnerability Assessment Concepts

1.1 What is a Vulnerability?

A vulnerability is a weakness, flaw, or gap in a system's design, implementation, configuration, or operational procedure that could be exploited by a threat actor to violate the system's security policy — causing unauthorized access, disclosure, modification, or disruption of data or service.

1.2 What is Vulnerability Assessment?

A systematic process of identifying, quantifying, and prioritizing (ranking) the vulnerabilities present in a system, network, or application. It answers "what weaknesses exist and how severe are they?" — distinct from penetration testing, which goes further to actually *exploit* findings and demonstrate real-world impact.

1.3 Vulnerability Assessment vs. Penetration Testing

Aspect	Vulnerability Assessment	Penetration Testing
Goal	Identify and prioritize weaknesses	Demonstrate real-world exploitability/impact
Depth	Broad, often automated	Deep, often manual, exploit-driven
Output	List of findings, ranked by severity	Narrative of how an attacker chains findings to achieve an objective
Frequency	Regular/continuous (e.g., monthly/quarterly)	Periodic, often annual or milestone-driven
Risk of disruption	Lower (mostly non-intrusive checks)	Higher (active exploitation can affect stability)

1.4 Types of Vulnerability Assessment

- **Network-based assessment**: identifies vulnerabilities in network infrastructure (routers, switches, firewalls) and exposed services.
- **Host-based assessment**: examines individual servers/workstations for missing patches, misconfigurations, weak permissions.
- **Application assessment**: reviews web/mobile/desktop applications for flaws such as injection, broken authentication, insecure configuration.
- **Database assessment**: checks database servers for misconfigurations, weak access controls, unpatched engines.
- **Wireless network assessment**: evaluates Wi-Fi configurations, encryption strength, rogue access points.

1.5 Vulnerability-Assessment Lifecycle

1. **Pre-assessment (planning/scoping)**: define scope, rules of engagement, assets in scope, and assessment windows.
2. **Assessment (scanning/testing)**: run automated scans and manual verification against the defined scope.
3. **Post-assessment (reporting/remediation)**: analyze results, prioritize by risk, document findings, and track remediation to closure.

1.6 Types of Vulnerability Scanning Approaches

- **Active scanning**: sends probes/requests directly to targets to elicit responses (most common approach; can be mildly intrusive).
- **Passive scanning**: observes existing network traffic without sending probes, lower risk of disruption but less thorough.
- **Credentialed (authenticated) scanning**: logs into the target with valid credentials to inspect patch levels, configuration, and installed software from the inside — far more accurate and comprehensive.
- **Non-credentialed (unauthenticated) scanning**: probes the target from the outside only, as an unauthenticated attacker would see it — useful for an external attacker's-eye view but less thorough.
- **External scanning**: performed from outside the organization's network perimeter.
- **Internal scanning**: performed from within the internal network, revealing exposure to an insider or post-breach attacker.

1.7 Challenges in Vulnerability Assessment

- Keeping scan signatures/plugins current against newly disclosed vulnerabilities.
- False positives (reported vulnerabilities that aren't actually exploitable) and false negatives (missed real vulnerabilities).
- Scanning coverage gaps (unknown/unmanaged assets — "shadow IT").
- Balancing scan intensity against the risk of disrupting production systems.
- Prioritizing a large volume of findings against limited remediation resources.

2. Vulnerability Assessment Solutions

2.1 Types of Vulnerability Assessment Solutions

- **Network/host vulnerability scanners**: automated tools that probe systems for known vulnerabilities (e.g., Nessus, OpenVAS, Qualys).
- **Web application scanners**: focused specifically on web vulnerabilities (e.g., Nikto, Burp Suite, Acunetix).
- **Cloud security posture management (CSPM)**: assesses cloud infrastructure configurations against security best practices.
- **Configuration/compliance scanners**: check systems against hardening benchmarks (e.g., CIS Benchmarks) rather than just known CVEs.
- **Static/dynamic application security testing (SAST/DAST)**: analyze application source code or running applications for security flaws.

2.2 Deployment Models

Model	Description
Product-based (on-premises)	Software installed and run within the organization's own infrastructure, giving full control over data and scan scheduling.
Service-based (SaaS/managed)	A cloud-hosted or third-party-managed scanning service; lower maintenance overhead but data leaves the organization's direct control.
Tree-based/Inference-based assessment	The scanner builds a logical model of the target (starts broad, then follows discovered details — e.g., detecting an OS, then only running checks relevant to that OS) to reduce unnecessary scanning.

2.3 Choosing a Vulnerability Assessment Solution — Key Considerations

- Breadth and freshness of the vulnerability/plugin database.
- Support for credentialed scanning across the organization's OS/application mix.
- Integration with ticketing/SIEM/patch-management systems for remediation workflows.
- Reporting flexibility (executive summaries vs. technical detail) and support for compliance frameworks.
- Scalability (number of assets, scan concurrency) and impact on production systems during scanning.
- False-positive rate and ease of result validation/tuning.

3. Vulnerability Scoring Systems

3.1 Why Scoring Matters

Raw vulnerability lists are not actionable without a consistent way to prioritize them — scoring systems translate technical characteristics of a vulnerability into a standardized severity rating so organizations can triage remediation effort.

3.2 CVSS (Common Vulnerability Scoring System)

The industry-standard open framework (maintained by FIRST.org) for rating the severity of vulnerabilities on a scale of **0.0 to 10.0**.

CVSS Metric Groups

Group	Purpose
Base Metrics	Intrinsic, constant characteristics of the vulnerability (e.g., Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, Confidentiality/Integrity/Availability impact).
Temporal Metrics	Characteristics that change over time (e.g., Exploit Code Maturity, Remediation Level, Report Confidence).
Environmental Metrics	Characteristics specific to a user's environment (e.g., modified base metrics, security requirements for confidentiality/integrity/availability in that context).

CVSS Severity Rating Scale (v3.x)

Score Range	Severity
0.0	None
0.1 - 3.9	Low
4.0 - 6.9	Medium
7.0 - 8.9	High
9.0 - 10.0	Critical

3.3 Other Vulnerability Databases and Scoring References

- **CVE (Common Vulnerabilities and Exposures)**: a standardized dictionary of publicly disclosed vulnerabilities, each with a unique identifier (e.g., `CVE-2024-12345`) — covered hands-on in Section 7.
- **CWE (Common Weakness Enumeration)**: a community-developed taxonomy of common *software/hardware weakness types* (the root-cause category), as opposed to CVE's specific instances — covered hands-on in Section 6.
- **National Vulnerability Database (NVD)**: the U.S. government repository that enriches each CVE entry with CVSS scores, CWE mappings, and affected-product (CPE) data.
- **CPE (Common Platform Enumeration)**: a standardized naming scheme for identifying applications, operating systems, and hardware, used to map vulnerabilities to specific affected products/versions.
- **EPSS (Exploit Prediction Scoring System)**: estimates the probability that a vulnerability will actually be exploited in the wild in the near term, complementing CVSS's severity-only view with a likelihood dimension.
- **KEV (Known Exploited Vulnerabilities Catalog)**: CISA's list of vulnerabilities confirmed to be actively exploited, often used to prioritize remediation above CVSS score alone.

3.4 Why CVSS Alone Isn't Enough

A high CVSS score reflects theoretical severity, not necessarily real-world risk to a specific organization. Mature vulnerability-management programs combine CVSS with asset criticality, exploit availability (EPSS/KEV), and compensating controls to prioritize remediation realistically.

4. Vulnerability Assessment Tools

4.1 Categories of Tools

- **General-purpose network/host scanners**: broad coverage of OS, service, and application vulnerabilities.
- **Web application scanners**: specialized for HTTP(S)-based vulnerabilities.
- **Configuration/compliance auditors**: check against hardening baselines rather than CVE lists.
- **Specialized scanners**: database-specific, wireless-specific, container/cloud-specific tools.

4.2 Notable Vulnerability Assessment Tools

Tool	Brief Description
Nessus (Tenable)	One of the most widely used commercial vulnerability scanners; extensive, frequently-updated plugin library covering network, OS, and application vulnerabilities, with strong credentialed-scan support.
OpenVAS (Greenbone Vulnerability Management)	A free, open-source vulnerability scanner and manager, forked from the last open Nessus release; widely used in labs and by organizations wanting an open alternative.
Qualys VMDR	Cloud-based vulnerability management platform offering continuous scanning, asset discovery, and prioritization at enterprise scale.
Nikto	A lightweight, open-source command-line web/CGI vulnerability scanner that checks web servers for thousands of known-dangerous files, outdated software versions, and misconfigurations.
Nexpose / InsightVM (Rapid7)	Vulnerability management platform with real-time risk scoring and integration into Rapid7's broader security suite.
Burp Suite	Primarily a web-application testing proxy/toolkit; its scanner component identifies web-specific vulnerabilities (injection, XSS, etc.).
OWASP ZAP	Free, open-source web application security scanner, popular for both automated and manual web app testing.
Nmap (with NSE vulnerability scripts)	Primarily a port scanner, but its Scripting Engine (NSE) includes vulnerability-detection scripts (`--script vuln`) for quick, targeted checks.

4.3 Choosing Between Tools (Practical Notes)

- **Nessus** is favored for depth, plugin freshness, and ease of use, but is commercial (with a limited free "Essentials" tier for small environments).
- **OpenVAS** is favored where an open-source, self-hosted, no-license-cost solution is required, at the cost of a steeper setup and somewhat less polished UX.
- **Nikto** is best used as a fast, pre-flight web-server/CGI check before a deeper, more comprehensive scan (e.g., OWASP ZAP or Burp Suite) — it is not a full web-application security testing replacement.

5. Vulnerability Assessment Reports

5.1 Purpose of the Report

The report is the primary deliverable of a vulnerability assessment — it must communicate risk clearly enough for both technical teams (to remediate) and leadership (to understand business risk and allocate resources) to act on it.

5.2 Typical Report Structure

1. **Executive Summary**: high-level overview of scope, overall risk posture, and key findings for non-technical stakeholders.
2. **Scope and Methodology**: what was tested, how, when, and with what tools/credentials.
3. **Findings Summary**: a prioritized table of vulnerabilities by severity (Critical/High/Medium/Low), often with counts and trend comparison to prior assessments.
4. **Detailed Findings**: for each vulnerability — description, affected assets, CVSS score/vector, evidence (screenshots/scan output), and business impact.
5. **Remediation Recommendations**: specific, actionable steps (patch version, configuration change, compensating control) per finding, ideally with target timeframes based on severity.
6. **Appendices**: raw scan output, asset inventory, glossary of terms.

5.3 Characteristics of a Good Report

- **Prioritized**, not just an exhaustive dump of every scanner finding — focuses attention on what matters most.
- **Actionable**: each finding pairs with a concrete remediation step, not just a description of the problem.
- **Accurate**: false positives are validated/removed before distribution to preserve credibility.
- **Reproducible**: enough detail (requests, parameters, screenshots) that a technical reader can confirm the finding independently.
- **Tracked over time**: supports trend analysis across repeated assessments to show whether risk is improving.

5.4 Common Reporting Formats/Standards

- Tool-native reports (Nessus `.nessus`/PDF/HTML, OpenVAS PDF/XML reports).
- Mapping to compliance frameworks (PCI DSS, HIPAA, ISO 27001) where required by regulation.
- Integration exports (CSV/JSON/XML) for ingestion into ticketing systems (Jira, ServiceNow) or GRC platforms.

6. Lab Walkthrough: Perform Vulnerability Research in CWE

6.1 Objective

Use the Common Weakness Enumeration (CWE) database to research a root-cause software weakness category and understand how it relates to real-world vulnerabilities.

6.2 About CWE

CWE (maintained by MITRE, at **https://cwe.mitre.org**) is a community-developed, categorized list of software and hardware weakness *types* — e.g., "Improper Neutralization of Special Elements used in an SQL Command" (SQL Injection) is **CWE-89**. Where CVE describes one specific, dated vulnerability instance, CWE describes the underlying pattern that many CVEs share.

6.3 Steps

1. Navigate to **https://cwe.mitre.org** in a browser.
2. Use the **Search CWE** box (or browse the **CWE List** by view, e.g., "Research Concepts" or "Software Development") to locate a weakness of interest — for example, search for `SQL Injection` or browse directly to **CWE-89**.
3. On the weakness's detail page, review:
   - **Description**: a plain-language explanation of the weakness.
   - **Extended Description**: deeper technical context.
   - **Common Consequences**: what security property is typically violated (Confidentiality, Integrity, Availability) and the technical impact.
   - **Potential Mitigations**: recommended secure-coding or architectural fixes mapped to the relevant development phase (e.g., "Architecture and Design," "Implementation").
   - **Observed Examples**: a short list of real CVE IDs that are concrete instances of this weakness class, directly linking the concept to Section 7's CVE research.
   - **Related CWEs**: parent/child/peer weaknesses in the taxonomy (e.g., CWE-89 is a child of the broader **CWE-943**, "Improper Neutralization of Special Elements in Data Query Logic").
4. Cross-reference the **CWE Top 25 Most Dangerous Software Weaknesses** list (published annually, also on cwe.mitre.org) to see which weakness categories are currently considered highest-impact/most-prevalent industry-wide.

6.4 Expected Result

A clear understanding of *why* a class of vulnerability occurs (root cause), what its typical impact is, and how to prevent it at the design/coding level — context that makes individual CVE findings (Section 7) far more actionable than a bare "patch this" instruction.

7. Lab Walkthrough: Perform Vulnerability Research in CVE

7.1 Objective

Research a specific, publicly disclosed vulnerability using its CVE identifier to understand affected products, severity, and available fixes.

7.2 About CVE

CVE (also maintained under MITRE's stewardship, with records published at **https://cve.org** and enriched at the **NVD**, **https://nvd.nist.gov**) assigns a unique identifier in the format `CVE-YYYY-NNNNN` to each individual, publicly disclosed vulnerability, enabling consistent reference across vendors, tools, and researchers.

7.3 Steps

1. Navigate to **https://cve.org** (CVE List search) or **https://nvd.nist.gov/vuln/search** (NVD, which adds CVSS/CWE enrichment).
2. Search either by:
   - A specific **CVE ID** if known (e.g., `CVE-2021-44228`, the Log4Shell vulnerability), or
   - **Keyword** (product/vendor name) to browse all CVEs affecting that product.
3. On the NVD detail page for a result, review:
   - **Description**: what the vulnerability is and the general attack scenario.
   - **CVSS Base Score and Vector String**: the severity rating and the specific metric values that produced it (tie this back to Section 3.2).
   - **CWE mapping**: which weakness category (Section 6) this specific CVE is an instance of.
   - **References**: links to vendor advisories, patches, and sometimes public proof-of-concept writeups.
   - **Known Affected Software Configurations (CPE)**: the exact product versions impacted.
4. Cross-check whether the CVE appears in **CISA's Known Exploited Vulnerabilities (KEV) catalog** (https://www.cisa.gov/known-exploited-vulnerabilities-catalog) — a strong signal that this vulnerability is being actively exploited and should be prioritized.
5. Identify the vendor's official patch/advisory and the remediation version, to be included in the assessment report's recommendations (Section 5).

7.4 Expected Result

A complete picture of one specific vulnerability — its severity, root cause (via CWE), affected versions, and the exact fix — ready to be matched against your own asset inventory to determine organizational exposure.

8. Lab Walkthrough: Perform Vulnerability Analysis Using OpenVAS

> ***Lab-only**: run only against systems in an isolated lab you own, or with explicit written authorization.*

8.1 Objective

Use OpenVAS (Greenbone Vulnerability Management) to scan a lab target and review the resulting vulnerability report.

8.2 Prerequisites

- A Linux host (or a dedicated distribution such as Kali Linux, which packages OpenVAS/GVM) with OpenVAS/GVM installed.
- A lab target machine (e.g., an intentionally-vulnerable VM such as Metasploitable2) on an isolated network segment.

8.3 Setup and Initial Sync

```
sudo apt update && sudo apt install openvas     # or: gvm
sudo gvm-setup        # downloads/syncs vulnerability feeds (NVTs, CERT, SCAP) - can take a while
sudo gvm-check-setup  # verifies the installation completed correctly
```

Setup generates an initial admin password, printed at the end of `gvm-setup` — record it for the web interface login.

8.4 Steps

1. Start the service and access the web UI:

   ```
   sudo gvm-start
   ```

   Then browse to **https://127.0.0.1:9392** and log in with the admin credentials from setup.

2. Define the scan target: **Configuration → Targets → New Target**. Enter the lab VM's IP address (or a CIDR range for multiple hosts) and, for a credentialed scan, attach SSH/SMB credentials under **Credentials**.

3. Create a scan task: **Scans → Tasks → New Task**. Give it a name, select the target just created, and choose a **Scan Config** (e.g., "Full and fast" for a balanced, comprehensive default).

4. Click the **Start** (play) icon on the task to launch the scan. Progress is shown as a percentage; scan duration depends on target count and scan config depth.

5. Once complete, open **Scans → Results** (or the task's report) to review findings, each listed with its **CVSS severity**, **affected host/port**, **CVE reference(s)** where applicable, and a **Solution** field describing the fix.

6. Filter/sort by severity to prioritize **Critical/High** findings first.

7. Export the report (**Report → Export**) in PDF or XML format for inclusion in your assessment documentation (Section 5).

8.5 Expected Result

A prioritized list of discovered vulnerabilities on the lab target, each mapped to a CVE/CVSS score and an actionable remediation note — directly populating a Section 5-style report.

9. Lab Walkthrough: Perform Vulnerability Scanning Using Nessus

> ***Lab-only**: run only against systems in an isolated lab you own, or with explicit written authorization. The free **Nessus Essentials** tier is limited to a small number of IP addresses, suitable for home-lab study.*

9.1 Objective

Install and run Nessus against a lab target, then interpret the scan results.

9.2 Setup

1. Register for a free **Nessus Essentials** activation code at Tenable's site, then download the installer for your OS.
2. Install the package (e.g., on Debian/Ubuntu):

   ```
   sudo dpkg -i Nessus-<version>-ubuntu.deb
   sudo systemctl start nessusd
   ```

3. Browse to **https://localhost:8834**, accept the self-signed certificate warning, and complete the setup wizard (create an admin account, enter the activation code; plugins will then download — this can take some time on first run).

9.3 Steps

1. Log in to the Nessus web UI and click **New Scan**.
2. Choose a scan template — for a general first pass, **Basic Network Scan** is typical.
3. Name the scan, and enter the lab target's IP address (or range) in the **Targets** field.
4. (Recommended for accuracy) Under **Credentials**, add SSH (Linux) or Windows credentials for the target to enable a **credentialed scan**, which checks actual patch levels and installed software rather than only external, network-visible indicators.
5. Save, then click the **Launch** (play) icon to start the scan.
6. Once complete, open the scan to view the results dashboard: findings are grouped by host and by **severity** (Critical/High/Medium/Low/Info), each using the Tenable Vulnerability Priority Rating (VPR) alongside CVSS.
7. Click into an individual finding to see its **Description**, **Synopsis**, **CVE references**, **CVSS Base Score/Vector**, **Plugin output** (the specific evidence Nessus found), and a **Solution** section.
8. Export the results: **Report → Export → PDF/CSV/Nessus** for inclusion in your assessment report.

9.4 Expected Result

A detailed, severity-ranked vulnerability list for the lab target, each finding backed by specific plugin evidence and a concrete remediation step — illustrating why credentialed scanning (Section 1.6) produces far more actionable results than an unauthenticated scan alone.

10. Lab Walkthrough: Perform Web Server/Application Vulnerability Scanning Using Nikto

> ***Lab-only***: *run only against a web server/application you own or are explicitly authorized to test.*

10.1 Objective

Use Nikto, a lightweight CLI web/CGI vulnerability scanner, to quickly identify outdated software, dangerous files, and common misconfigurations on a lab web server.

10.2 Setup

Nikto ships by default on Kali Linux; otherwise install via:

```
sudo apt install nikto
```

or clone directly from its GitHub repository and run the Perl script.

10.3 Basic Usage

```
nikto -h http://<target-ip-or-hostname>
```

Common useful flags:

```
nikto -h http://<target> -p 80,443          # specify ports explicitly
nikto -h https://<target> -ssl              # force SSL/TLS
nikto -h http://<target> -o report.html -Format htm   # save an HTML report
nikto -h http://<target> -Tuning x 6         # tune which test categories to run (e.g., exclude DoS-prone
checks)
```

10.4 Steps

1. Confirm the lab target web server is reachable (`curl -I http://<target>`).
2. Run a baseline scan: `nikto -h http://<target>`.
3. Review the console output as it streams: each line flags a specific finding, e.g.:
   - Outdated server banner/version (e.g., an old Apache/PHP version with known CVEs)
   - Presence of dangerous or default files (e.g., leftover `phpinfo.php`, backup files like `index.php.bak`)
   - Missing security headers (e.g., `X-Frame-Options`, `Content-Security-Policy`)
   - Directories allowing listing, or default/admin login pages exposed
   - Outdated or vulnerable CGI scripts (Nikto's original specialty, per the "CGI scanner" name)
4. Save a formatted report for documentation: `-o report.html -Format htm` (or `.csv` / `.xml` / `.txt` as needed).
5. Treat each finding as a **lead for manual verification** rather than a confirmed exploit — Nikto is intentionally broad and noisy (it does not attempt exploitation), so cross-check findings against the CVE/CWE research steps (Sections 6-7) and, where warranted, a deeper tool such as OWASP ZAP or Burp Suite.

10.5 Expected Result

A fast, text-based list of web-server-level hygiene issues (outdated versions, dangerous files, missing headers) suitable as the first pass of a web-application assessment, to be followed by deeper application-logic testing tools for anything Nikto cannot evaluate (business logic, authentication flaws, etc.).

Quick Revision Summary

#	Topic	One-line takeaway
1	Vulnerability Assessment Concepts	Identify & prioritize weaknesses; distinct from (and usually a precursor to) penetration testing.
2	Vulnerability Assessment Solutions	Network, web, cloud, and compliance scanners; on-prem vs. SaaS deployment trade-offs.
3	Vulnerability Scoring Systems	CVSS (0-10 severity) plus CVE/CWE/NVD/EPSS/KEV for context and real-world exploitation likelihood.
4	Vulnerability Assessment Tools	Nessus/OpenVAS (network), Nikto/ZAP/Burp (web) - each with a different depth/scope niche.
5	Vulnerability Assessment Reports	Executive summary + prioritized, actionable, validated findings drive remediation.
6	CWE Research	Root-cause weakness taxonomy (e.g., CWE-89 SQL Injection); links to real CVE examples.
7	CVE Research	Specific disclosed vulnerability instances; check CVSS score, CWE mapping, and CISA KEV status.
8	OpenVAS Lab	Free/open-source scanner; target → task → launch → severity-ranked report with CVE/solution.
9	Nessus Lab	Industry-standard scanner; credentialed scans give far more accurate, actionable results.
10	Nikto Lab	Fast CLI web/CGI hygiene scan (outdated versions, dangerous files); a first pass, not a full app test.